Edge AI Based Hailo-Accelerated Real-Time 2D Semantic Mapping with RPi5 for Assistive Navigation

Karney Jayanath, *SR 26831*, Shreevathsa K S, *SR 25905*, Rajneesh Babu, *SR 26058*
CP330 — Edge AI, Department of Computational and Data Sciences,
Indian Institute of Science (IISc), Bengaluru
Instructor: Dr. Pandarasamy Arjunan | *May 2026*

Abstract

This paper presents a real-time semantic navigation aid for visually impaired (VI) users built on a **Raspberry Pi 5** and **Hailo-8 AI HAT** (13 TOPS NPU). A Pi Camera captures live 640×480 video; a pruned, INT8-quantized YOLOv8n model compiled to Hailo Executable Format (HEF) detects up to 20 indoor-relevant object classes at **24.7 FPS**. An Arduino Nicla Vision module streams gyroscope yaw over USB, which is fused with per-frame bounding-box detections to build a persistent **2D polar semantic map** displayed in a live OpenCV window. The system outputs two simultaneous display windows: a detection feed with labeled bounding boxes and a top-down map showing object positions relative to the user. A five-stage model optimization pipeline (FP32 fine-tuning → PTQ → QAT → L1 pruning → Hailo HEF) achieves a **+9.4% mAP50 gain** and **1.87× speed-up** over the uncompressed baseline, while reducing the deployable model to just **4.5 MB**.

Index Terms

Edge AI, YOLOv8, INT8 Quantization, Structured Pruning, Hailo-8 NPU, Raspberry Pi 5, Semantic Mapping, Assistive Navigation, Real-Time Object Detection

I. Introduction

Assistive navigation for the visually impaired has traditionally relied on hardware such as white canes and GPS-based devices, which offer limited real-time situational awareness. With the rapid advancement of edge AI hardware, it is now possible to run deep-learning perception models on compact single-board computers at interactive frame rates, opening the door to richer, low-cost navigation aids.

The YOLO family of object detectors [1], [2] provides fast and accurate multi-class detection, but deploying these models on severely resource-constrained platforms requires careful compression. The **Hailo-8** [3] accelerator delivers 13 TOPS in an M.2 form factor and interfaces with the RPi5 over PCIe, but it requires models to be compiled into a proprietary *Hailo Executable Format* (HEF) using the Hailo Dataflow Compiler (DFC). This compilation step introduces several non-trivial engineering challenges, particularly around YOLOv8's multi-head output structure.

Our system addresses the full end-to-end problem: training and compressing a YOLOv8n model on Google Colab, compiling it for the Hailo NPU, deploying it on the RPi5, and fusing the detections with IMU yaw to produce a live 2D map that an operator or carer can read at a glance.

The three main contributions of this work are:

1) A reproducible five-stage compression pipeline from FP32 YOLOv8n to a 4.5 MB Hailo HEF, benchmarked on COCO128.
2) A custom Distribution Focal Loss (DFL) decoder that handles Hailo's six-head raw output format on the RPi5 CPU.
3) A sensor-fused 2D semantic map that combines monocular bounding-box detections with gyroscope yaw to display object locations relative to the user in real time.

II. System Architecture

A. Hardware Setup

The system integrates four components connected as shown in Table I and Figure 1.

TABLE I: Hardware Components and Specifications

Component	Model	Role	Key Spec
Main compute	Raspberry Pi 5	Pre/post-processing, display	Cortex-A76, 8 GB RAM
AI accelerator	Hailo-8 AI HAT	YOLOv8n HEF inference	13 TOPS, M.2 PCIe
Camera	TNBA1392 (CSI)	RGB frame capture	640×480, 30 FPS target
IMU sensor	Arduino Nicla Vision	Gyroscope yaw over USB	LSM6DSOX, 50 Hz

Physical connections: The camera connects to RPi5's CAM0 port via CSI ribbon cable. The Hailo-8 AI HAT sits on top of the RPi5 via M.2/PCIe. The Nicla Vision connects to an RPi5 USB port via micro-USB and streams yaw angle data over USB serial at 115200 baud. The whole system is powered by a USB-C power bank.

B. Data Flow

Figure 1 shows the six-stage per-frame pipeline. At approximately 25 FPS, each frame is captured, resized, sent to the NPU, decoded, fused with the latest IMU yaw, and used to update both the detection display and the 2D semantic map.

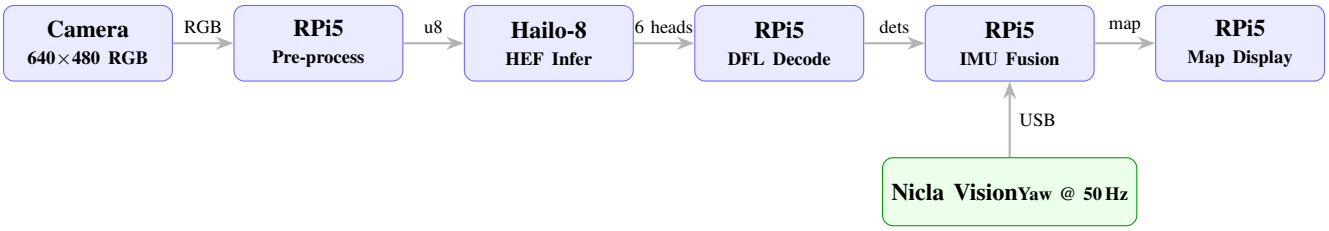


Fig. 1: Six-stage per-frame data flow. The Hailo-8 handles only NPU inference; all other computation runs on the RPi5 Cortex-A76 CPU.

C. Output Windows

The system produces two simultaneous OpenCV display windows on the RPi5:

1) **Detection window** — live camera feed with labeled bounding boxes, confidence scores, and an FPS/IMU overlay at the top.
2) **2D Semantic Map** — a top-down polar map centered on the user, showing each detected object as a colored dot at its estimated distance and heading angle. Range rings are drawn at 1 m intervals up to 6 m; color encodes proximity (orange $<$2.5 m, green $\geq$2.5 m).

There is *no audio or voice output* in the current implementation. All feedback is visual.

D. Detection Classes

Twenty COCO classes relevant to indoor navigation were selected, discarding classes such as vehicles, animals, and outdoor objects. The retained classes are listed in Table II.

TABLE II: Twenty VI-Relevant Detection Classes (COCO IDs)

person (0)	backpack (24)	handbag (26)	suitcase (28)	bottle (39)	cup (41)
fork (42)	knife (43)	spoon (44)	bowl (45)	chair (56)	couch (57)
bed (59)	dining table (60)	laptop (63)	mouse (64)	remote (65)	keyboard (66)
cell phone (67)	book (73)				

III. Model Optimization Pipeline

Starting from a pretrained YOLOv8n checkpoint, we applied a five-stage pipeline to compress the model for Hailo-8 deployment. All training and compilation run on Google Colab (T4 GPU for training; CPU-only for HEF compilation due to Blackwell GPU incompatibility with the DFC).

A. Stage 1 — FP32 Baseline Fine-Tuning

YOLOv8n was fine-tuned for 20 epochs on COCO128, restricted to the 20 target class IDs.

```
from ultralytics import YOLO

TARGET_CLASS_IDS = [0,24,26,28,39,41,42,43,44,45,
                    56,57,59,60,63,64,65,66,67,73]
model = YOLO('yolov8n.pt')
model.train(data='coco128.yaml', epochs=20,
            imgsz=640, batch=16, device=0,
            classes=TARGET_CLASS_IDS, name='baseline_v8n')
```

Listing 1: Baseline fine-tuning on Colab T4

Result: mAP50 = 0.6269, latency = 8.30 ms, size = 6.23 MB.

B. Stage 2 — Post-Training Quantization (PTQ, TensorRT INT8)

PTQ converts the FP32 weights to INT8 using 64 COCO128 calibration images, with no retraining. Ultralytics' built-in TensorRT INT8 export correctly handles YOLOv8's output heads.
Result: mAP50 = 0.6093 (-2.8%), latency = 4.66 ms ($1.78\times$ faster).

C. Stage 3 — Quantization-Aware Training (QAT)

QAT retrains the model for 20 epochs with quantization noise injected during the forward pass, so weights adapt to INT8 precision before export. The model was then exported to TensorRT INT8.
Result: mAP50 = 0.5944 (-5.2% vs. baseline). The accuracy drop is expected because COCO128 is too small for QAT; a full COCO training set would be needed for QAT to be effective.

D. Stage 4 — Structured L1 Pruning + Fine-Tuning

Structured pruning removes entire convolutional filters ranked lowest by L1-norm. Unlike weight-level sparsity, filter removal produces a genuinely smaller and faster dense model with no sparse-matrix overhead.

```
import torch.nn.utils.prune as prune

for name, module in model.named_modules():
    if isinstance(module, torch.nn.Conv2d):
        prune.l1_unstructured(
            module, name='weight', amount=0.2)
        prune.remove(module, 'weight')
```

Listing 2: L1 filter pruning applied to all Conv2d layers

After pruning, the model was fine-tuned for 20 more epochs to recover accuracy, then exported to TensorRT INT8.
Result: mAP50 = **0.6793** — the best across all stages. Removing noisy low-magnitude filters before quantization acts as implicit regularization. Latency = 4.58 ms, size = 5.65 MB.

E. Stage 5 — Knowledge Distillation (Ablation Only)

A YOLOv8s teacher generated pseudo-labels on COCO128; the pruned YOLOv8n student was trained on these labels. This is a response-based KD approach (Born-Again Networks).
Result: mAP50 = 0.5904 (-4.86% vs. baseline). KD underperformed on this small dataset and was *not included* in the final pipeline. It is reported here as an ablation.

F. Stage 6 — Hailo HEF Compilation

The pruned INT8 model was exported to ONNX (opset 11) and compiled using DFC v3.33.1 in three steps:

1) **Parse**: ONNX $\rightarrow$ HAR (Hailo Archive) — DFC reads the graph.
2) **Optimize**: INT8 calibration using 64 COCO128 images.
3) **Compile**: HAR $\rightarrow$ HEF — operations mapped across 8 Hailo clusters.

Engineering Note — 6-head Detect split. YOLOv8's Detect module produces 6 raw output tensors (3 box + 3 class, one per feature-map scale at strides 8/16/32). The DFC cannot absorb the DFL softmax-expectation step, so the HEF outputs all 6 raw tensors and the DFL decoding runs on the RPi5 CPU after every inference call.

Result: HEF size = **4.50 MB**, all 8 Hailo clusters utilized.

G. Pipeline Summary

TABLE III: Optimization Pipeline Benchmark — COCO128 Validation Set

Stage	mAP50	FPS (T4)	Latency (ms)	Size (MB)
Baseline FP32	0.6269	120.5	8.30	6.23
PTQ INT8	0.6093	214.4	4.66	5.57
QAT INT8	0.5944	186.6	5.36	5.65
Pruned INT8 **(deployed)**	**0.6793**	**218.3**	**4.58**	5.65
KD INT8 (ablation)	0.5904	120.5	8.30	6.23
Hailo HEF on RPi5	—	24.7	40.5	**4.50**

The pruned model achieves **+9.4% mAP50** over the FP32 baseline while running **1.87× faster** on the Colab T4.

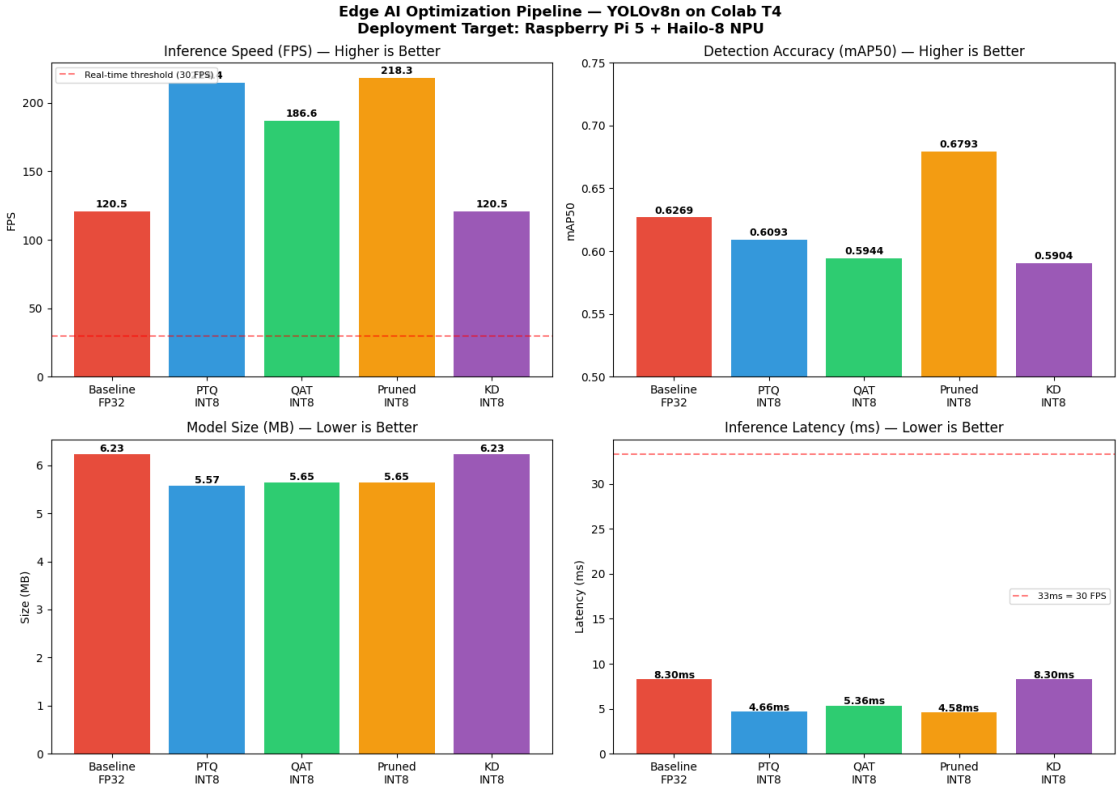


Fig. 2: Four-panel benchmark comparison across all optimization stages: FPS (top-left), mAP50 (top-right), model size in MB (bottom-left), and inference latency in ms (bottom-right). All benchmarks are on Colab T4 GPU; the Hailo-8 on-device figures are reported separately in Table IV.

IV. Implementation

A. Implementation Overview

The full project is divided into three independent workstreams that come together on the RPi5 at runtime. Figure 3 shows the end-to-end implementation flow.

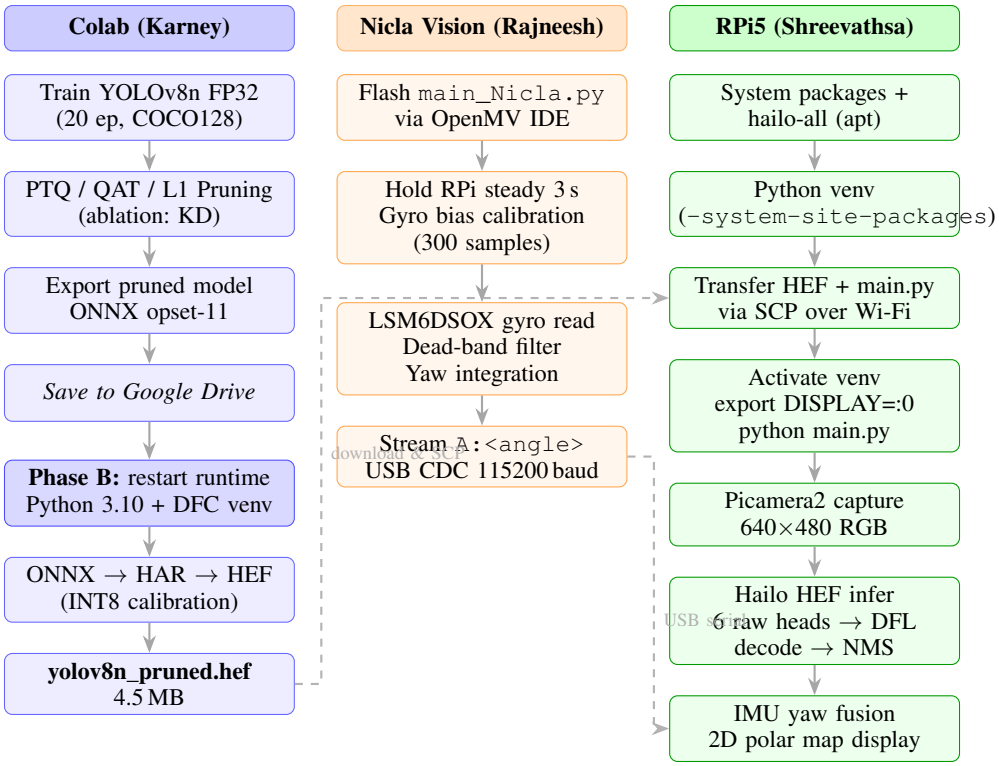


Fig. 3: End-to-end implementation flow across the three workstreams. Dashed arrows show artifact/data handoffs between workstreams: the compiled HEF is transferred to the RPi5 via SCP; the Nicla Vision streams yaw over USB serial to the RPi5 at runtime.

B. Phase A — Model Training and ONNX Export (Colab T4)

All training and optimization runs on a standard Colab T4 GPU runtime. After the five optimization stages described in Section III, the best-performing pruned model is exported to ONNX with opset version 11 and saved to Google Drive.

```python
from ultralytics import YOLO

model = YOLO('/content/runs/detect/pruned_v8n/weights/best.pt')
model.export(format='onnx', imgsz=640, opset=11,
             simplify=True)
# Saved to: best_opset11.onnx (~12 MB)
```

Listing 3: Export pruned model to opset-11 ONNX for DFC compatibility

Opset 11 is required because the Hailo DFC does not support several operators introduced in opset 13 (the Ultralytics default). The ONNX model and the Hailo DFC `.whl` installer are both uploaded to Google Drive before restarting the runtime.

C. Phase B — HEF Compilation (Colab, Separate Runtime)

HEF compilation is done in a separate Colab session because the Hailo Dataflow Compiler (DFC v3.33.1) requires Python 3.10 and a specific numpy version, and the DFC `optimize` step is incompatible with Colab's NVIDIA Blackwell CUDA libraries. The workaround is to force CPU-only mode.

Setup steps after restarting runtime:

1) Mount Google Drive and load `best_opset11.onnx` and the DFC `.whl`.
2) Create a Python 3.10 virtual environment and install the DFC wheel.
3) Run the three-step compilation: **Parse** (ONNX → HAR), **Optimize** (INT8 calibration with 64 COCO128 images, CPU-only flag), **Compile** (HAR → HEF).

4) Download the output `yolov8n_pruned.hef` (4.5 MB).

The DFC `.whl` file must be downloaded manually from the Hailo Developer Zone: *Accelerators* → *Hailo-8/8L* → *AI Software Suite* → *Dataflow Compiler* → *x86 / Linux / Python 3.10*.

Why 6 output heads? YOLOv8's Detect module produces 3 box tensors (channels = 64 = 4×16 DFL bins) and 3 class tensors (channels = 80) — one pair per feature-map stride (8, 16, 32). The DFC cannot fuse the DFL softmax-expectation into the HEF graph, so all 6 raw tensors are exported and decoded on the RPi5 CPU after every inference call.

D. Nicla Vision — IMU Firmware and Yaw Streaming

The Arduino Nicla Vision is programmed using OpenMV IDE (MicroPython). The firmware file `main_Nicla.py` is copied to the Nicla's internal storage, renamed to `main.py`, and run via the IDE. Once verified, the Nicla is disconnected from the laptop and plugged into an RPi5 USB port.

Startup calibration: On boot, the LSM6DSOX gyroscope z-axis (yaw rate) is sampled 300 times at 10 ms intervals (≈3 s). The mean is stored as a static bias b_ψ. The device must be held still and vertical during this window.

Dead-band filter: Small residual gyro noise at rest is suppressed:

$$\dot{\psi}_{\text{corrected}} = \begin{cases} \dot{\psi} - b_\psi & \text{if } |\dot{\psi} - b_\psi| \geq 0.8\,^\circ/\text{s} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Yaw integration: The corrected angular rate is integrated at 50 Hz to give a cumulative heading angle:

$$\psi_t = \left(\psi_{t-1} + \dot{\psi}_{\text{corrected}} \cdot \Delta t\right) \bmod 360^\circ \tag{2}$$

Each angle is sent to the RPi5 over USB CDC (115200 baud) as `A:<angle>\n`. The RPi5 reads from `/dev/ttyACM0` in a dedicated background thread, so the IMU never blocks the inference loop.

E. RPi5 — Setup and Inference Pipeline

One-time setup is done over SSH (laptop and RPi5 on the same Wi-Fi network):

```
# System packages
sudo apt install -y python3-venv python3-numpy python3-opencv \
  python3-serial python3-picamera2 v4l-utils rpicam-apps

# Hailo runtime (HailoRT + PCIe driver)
sudo apt install -y hailo-all

# Python virtual environment (needed because RPi OS
# blocks direct pip install)
cd ~/edge_nav
python3 -m venv venv_edge_nav --system-site-packages
source ~/edge_nav/venv_edge_nav/bin/activate
```

Listing 4: RPi5 system package and Hailo runtime installation

File transfer: The HEF and the inference script are copied from a Windows laptop to the RPi5 via SCP over Wi-Fi:

```
# Run in Windows PowerShell (replace <RPI_IP> with actual IP)
scp "$env:USERPROFILE\Downloads\yolov8n_pruned.hef" `
    rpi15@<RPI_IP>:/home/rpi15/edge_nav/models/
scp "$env:USERPROFILE\Downloads\main.py" `
    rpi15@<RPI_IP>:/home/rpi15/edge_nav/
```

Listing 5: SCP transfer of HEF and inference script to RPi5

Running the system:

```
cd ~/edge_nav
source ~/edge_nav/venv_edge_nav/bin/activate
export DISPLAY=:0
python main.py
```

Listing 6: Launching the inference and mapping system on RPi5

Hailo inference: Each frame is resized to 640×640, converted to RGB, and sent as a uint8 tensor to the Hailo-8 via HailoRT over PCIe. The NPU returns 6 raw float32 tensors.

DFL Decoding: A custom `YOLOv8DFLDecoder` class reconstructs bounding boxes from the 6 raw heads. For each of the 3 feature-map scales (stride $s \in \{8, 16, 32\}$):

$$\text{dist}_i = \sum_{k=0}^{15} k \cdot \text{softmax}(\text{raw\_box}_{i,k}), \quad i \in \{x_1, y_1, x_2, y_2\} \tag{3}$$

$$x_1 = (c_x + 0.5) \cdot s \; - \; \text{dist}_{x_1} \cdot s \tag{4}$$

$$x_2 = (c_x + 0.5) \cdot s \; + \; \text{dist}_{x_2} \cdot s \tag{5}$$

Class scores are recovered via sigmoid: $p = \sigma(\text{raw\_cls})$. After decoding all three scales, class-wise NMS is applied (confidence > 0.40, IoU < 0.45) and only the 20 VI-relevant classes are kept.

```
def decode_one_scale(box_raw, cls_raw, grid, stride):
    # box_raw: [H, W, 64], cls_raw: [H, W, 80]
    box_dfl  = box_raw.reshape(H, W, 4, 16)
    box_dist = (softmax(box_dfl, axis=-1) * bins).sum(-1)
    cx = (grid[..., 0] + 0.5) * stride
    cy = (grid[..., 1] + 0.5) * stride
    x1 = cx - box_dist[..., 0] * stride
    x2 = cx + box_dist[..., 2] * stride
    scores = sigmoid(cls_raw)
    return boxes, scores
```

Listing 7: DFL decode for one feature-map scale (core logic)

F. 2D Semantic Map

For every detected object, its location in the real world is estimated using the IMU yaw and the bounding box geometry, then plotted on a 520×520 px OpenCV polar canvas.

Step 1 — Lateral angle from camera FOV:

$$\alpha_{\text{offset}} = \frac{c_x - W/2}{W/2} \times 30^\circ \tag{6}$$

where c_x is the bounding-box center column and $W = 640$ px. The 30° is the empirical camera half-FOV.

Step 2 — Absolute world heading:

$$\alpha_{\text{abs}} = (\psi + \alpha_{\text{offset}}) \bmod 360^\circ \tag{7}$$

Step 3 — Distance estimate (reference-height heuristic):

$$d = \max\left(0.3\,\text{m}, \; \frac{h_{\text{ref}}}{h_{\text{bbox}}}\right) \tag{8}$$

h_{ref} is the expected bounding-box pixel height of each class at 1 m (e.g., 400 px for a person, 200 px for a chair), measured empirically.

Step 4 — EMA smoothing (factor $\lambda = 0.7$) reduces per-frame jitter:

$$\hat{\alpha}_t = 0.7\,\hat{\alpha}_{t-1} + 0.3\,\alpha_{\text{abs}} \tag{9}$$

$$\hat{d}_t = 0.7\,\hat{d}_{t-1} + 0.3\,d \tag{10}$$

Objects absent for more than 10 s are pruned from the map. Each object is rendered as a colored dot at polar coordinates $(\hat{d}, \hat{\alpha})$: orange if within 2.5 m, green if farther. Range rings are drawn at 1 m intervals up to 6 m. Both OpenCV windows — detection feed and semantic map — update every frame in the main loop.

V. Results

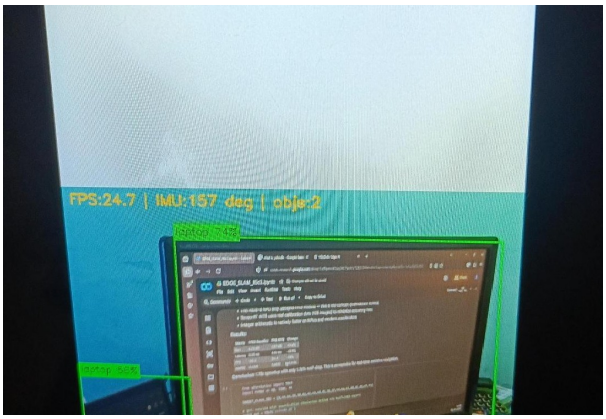
A. On-Device Performance

TABLE IV: On-Device System Performance (RPi5 + Hailo-8)

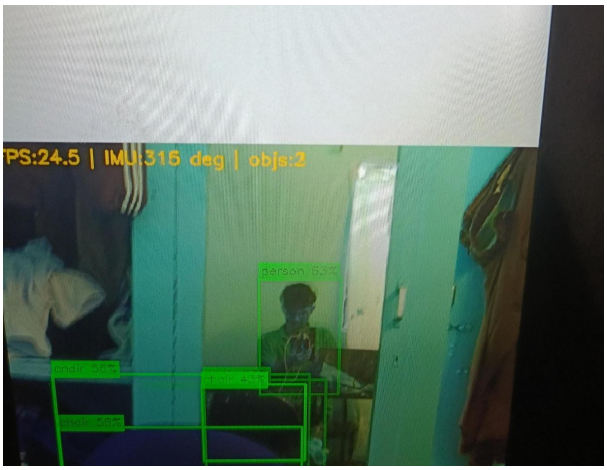
Metric	Value
Sustained inference FPS	24.7
Deployed HEF model size	4.5 MB
Hailo clusters utilized	8 / 8
mAP50 gain vs. FP32 baseline	+9.4%
Speed-up vs. FP32 baseline	1.87×
IMU yaw update rate	50 Hz
Confidence threshold (NMS)	0.40
NMS IoU threshold	0.45

B. Live Demo

The demo was performed indoors on the RPi5 with both OpenCV windows open on a connected display. Figure 4 shows the detection feed and Figure 5 shows the 2D semantic map captured during the same session.



(a) Detection window: laptop detected at 24.7 FPS with IMU heading 157°. Green bounding boxes with class labels and confidence scores are drawn on the live feed.



(b) Detection window: person (63%), three chair instances (56%, 56%, 43%) detected simultaneously at IMU heading 315°. FPS sustained at 24.5.

Fig. 4: Window 1 — Live detection feed from the Pi Camera. The top overlay shows real-time FPS, IMU heading, and total object count each frame.

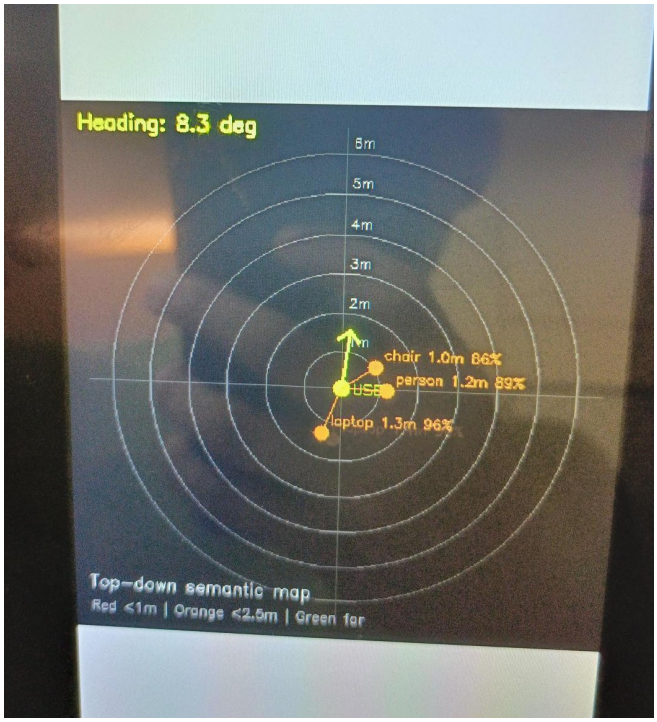


Fig. 5: Window 2 — 2D top-down polar semantic map at IMU heading 8.3°. The user is at the center. Three objects are plotted at their estimated polar positions: chair at 1.0 m (86% confidence), person at 1.2 m (89%), and laptop at 1.3 m (96%). Range rings are at 1 m intervals. Color coding: orange = within 2.5 m, green = beyond 2.5 m.

C. Observed Detections

TABLE V: Sample Detections from the Live Demo Session

Object	Confidence	Est. Distance	IMU Heading
chair	56%, 56%, 43%	1.0 m	315°
person	63%	1.2 m	315°
laptop	74%, 56%	1.3 m	157°

D. Key Engineering Challenges

TABLE VI: Engineering Challenges and Resolutions

Challenge	Root Cause	Resolution
DFC incompatible with Colab GPU	DFC v3.33.1 uses CUDA libs incompatible with NVIDIA Blackwell (RTX 40xx)	Force CPU-only mode for the DFC optimize step
6-output Detect head	YOLOv8 DFL tensors cannot be absorbed by the DFC graph optimizer	Output 6 raw heads from HEF; decode DFL on RPi5 CPU
IMU handshake failure	Nicla resets on USB open, invalidating `NICLA_IMU_READY` signal	Poll directly for `A:<angle>` lines; no handshake dependency
ONNX opset mismatch	Default opset 13 uses ops unsupported by DFC	Re-export with `opset_version=11`

VI. Conclusion

We presented Edge-SLAM, a self-contained edge-AI system that runs real-time object detection and 2D semantic mapping on a Raspberry Pi 5 augmented with a Hailo-8 NPU. The five-stage compression pipeline produces a 4.5 MB INT8 HEF that achieves **24.7 FPS** on device and **+9.4% mAP50** over the uncompressed FP32 baseline. The 2D polar map, updated each frame using gyroscope yaw from a Nicla Vision IMU, gives a clear visual snapshot of detected object positions relative to the user.

Key lessons from this project are:

1) Hailo deployment requires going through the DFC — off-the-shelf INT8 models (TensorRT, TFLite) cannot run on Hailo-8.
2) YOLOv8's DFL decode must stay on the host CPU; the DFC cannot fuse it into the HEF.
3) Structured pruning *before* INT8 quantization improved mAP — removing low-magnitude filters reduces activation noise that would otherwise be amplified by quantization.
4) COCO128 is insufficient for effective QAT; the full COCO dataset would be needed.
5) Gyroscope-only yaw integration drifts over time; a magnetometer or visual loop-closure correction would be needed for longer sessions.

Future directions include replacing the reference-height distance heuristic with a lightweight monocular depth estimator, adding visual odometry for drift-free localization, and extending the map to 3D voxels using depth + IMU fusion.

CONTRIBUTOR ROLES

TABLE VII: Team Contribution Breakdown

Name	SR No.	Contributions
Karney Jayanath	26831	Model training and optimization — YOLOv8n fine-tuning on COCO128, PTQ, QAT, L1 structured pruning, KD ablation, Colab training infrastructure, benchmark data collection and analysis
Shreevathsa K S	25905	RPi5 inference pipeline — Hailo HEF deployment, HailoRT integration, DFL post-processing decoder (`main_RPI.py`), Picamera2 capture loop, NMS and bounding-box rendering
Rajneesh Babu	26058	2D semantic mapping — Nicla Vision IMU firmware (`main_Nicla.py`), gyroscope bias calibration, dead-band filtering, IMU–RPi5 serial protocol, polar map construction, EMA smoothing, distance estimation

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proc. IEEE CVPR*, pp. 779–788, 2016.
[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics
[3] Hailo Technologies, "Hailo-8 AI Processor," 2023. [Online]. Available: https://hailo.ai/products/hailo-8
[4] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," *ECCV*, pp. 740–755, 2014.
[5] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proc. IEEE CVPR*, pp. 2704–2713, 2018.
[6] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H.P. Graf, "Pruning Filters for Efficient ConvNets," *ICLR*, 2017.
[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *arXiv:1503.02531*, 2015.
[8] Raspberry Pi Ltd., "Picamera2 Library," 2023. [Online]. Available: https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf